

02：区域压缩索引如何工作

1. 什么是索引

索引是一种用额外存储或预计算换取查询速度的数据结构。日常例子是书末索引：读者不必从第一页逐页查找某个术语，而可以先从索引定位页码。

本项目的索引不是保存所有 OD 的答案。它保存的是某些道路区域的精确内部穿越距离，让许多不同 OD 都能复用同一批预计算结果。

2. 什么是区域

区域是一组道路节点。本项目要求候选区域是**连通的**：忽略道路方向后，区域内任意部分不能成为完全孤立的小岛。

当前固定候选区域都包含 512 个节点。这个数字不是说 Porto 被平均切成 512 节点的块；而是从一个种子节点开始，用 BFS 向周围生长，直到收集 512 个节点。

3. BFS 是什么

BFS 是 Breadth-First Search，中文叫广度优先搜索。它按“离种子节点的图跳数”一圈圈扩张：

1. 先收下种子节点；
2. 再收下种子的邻居；
3. 再收下这些邻居尚未访问的邻居；
4. 直到区域达到指定节点数。

候选生成时把入邻居和出邻居合并，因此生长形状依据无向连通关系；但后续 shortcut 和查询仍严格使用原始有向边。

4. 种子节点是什么

种子节点只是 BFS 生长的起点，不代表它一定是区域最重要的节点。改变种子会改变 BFS 最先遇到哪些道路，从而产生不同区域。

第二版正式候选池在全图道路节点上使用固定随机种子，不读取历史 OD。这样候选空间不会提前偏向热点，NBFNet、MLP 和 Proxy 都在同一批中性候选上比较。详细生成过程在 06：固定候选区域 中解释。

5. 边界节点与内部节点

区域中的节点分两类：

- **边界节点**：至少有一条入边或出边连接到区域外；
- **内部节点**：属于区域，但所有直接相连的道路节点都在区域内。

还可以加上第三类：

- **外部节点**：不属于任何被压缩区域。

代码分别用 `NODE_BOUNDARY`、`NODE_INTERNAL` 和 `NODE_OUTSIDE` 表示三种状态。

直观示例：

```
区域外 X → [边界 A → 内部 B → 内部 C → 边界 D] → 区域外 Y
```

如果一条路线只是从 A 进入区域、从 D 离开区域，那么在线查询没有必要重新展开 B、C。可以提前保存 A 到 D 的精确内部距离。

6. Shortcut 是什么

Shortcut 可以译为“捷径边”。它不是凭空创造一条更短道路，而是把区域内部已经存在的最短路线折叠成一条等价边。

假设区域内：

```
A → B, 长度 3
B → C, 长度 4
C → D, 长度 5
```

如果 A 和 D 是边界节点，那么可建立：

```
A → D, shortcut 权重 12
```

权重 12 等于区域内 A 到 D 的精确最短距离。在线搜索走 shortcut，与展开 A、B、C、D 得到的距离完全相同，只是少处理内部节点。

有向图中 A 到 D 和 D 到 A 必须分别计算。即使 A 能到 D，也不代表 D 能到 A。

7. Shortcut 如何离线计算

对一个区域，代码会：

1. 找出全部边界节点；
2. 依次把每个边界节点当作起点；
3. 运行“受限 Dijkstra”，只允许访问当前区域中的节点；
4. 记录它到其他每个可达边界节点的精确距离；
5. 为每一对可达的有向边界节点创建 shortcut。

实现位于 `src/compression_index.py` 的 `build_compression_index` 和 `_restricted_dijkstra`。

如果一个区域边界很多，潜在 shortcut 数会接近边界数的平方，所以区域边界形状直接影响索引存储成本。

8. 什么叫“物化压缩图”

“物化”表示真正构造并存储一个可查询的新图，而不是查询时临时想象区域被压缩。

构造压缩图时：

1. 删除所有被压缩区域的内部节点；
2. 保留区域边界节点；
3. 保留所有区域外节点；
4. 保留连接这些剩余节点的原始边；
5. 加入离线计算的 shortcut；
6. 同一对节点存在多条边时，只保留权重最小的一条。

这样在线双向 Dijkstra 看到的是一个节点更少、但距离语义不变的图。

9. 为什么距离仍然精确

考虑一条原图最短路线。它每次进入一个被压缩区域时，必然从某个边界节点进入，并从某个反过来，压缩图中的每条 shortcut 都对应原图区域内一条真实路径，所以压缩图不可能凭空产生比原图更短的非合法路线。

因此，在端点处理正确、区域不重叠且 shortcut 精确时，压缩图最短距离应与原图一致。

项目仍对每条实验查询做距离校验，因为正确性不能只靠理论假设。

10. 为什么区域不能随便重叠

当前 `build_compression_index` 拒绝把重叠区域同时物化，因为同一个节点若同时属于两个区域，节点应删除还是保留、属于哪个边界系统、shortcut 如何组合都会产生歧义。

需要区分两个场景：

- **候选池中允许不同候选重叠**：因为阶段 1 每次只单独评测一个候选；
- **最终同时物化的区域集合不能重叠**：当前压缩索引实现要求如此。

这是“候选之间允许重叠”与“部署集合禁止重叠”看似矛盾、其实不同的原因。

11. 查询端点在区域外或边界上

如果起点和终点都存在于压缩图中，也就是它们不是被删除的内部节点，那么直接在压缩图上运行双向 Dijkstra 即可。

这时搜索可以按需使用 shortcut 穿过区域。

12. 查询端点恰好是内部节点时会怎样

内部节点已经从压缩图删除，不能直接作为压缩图搜索起点或终点。

第一版和更早的实现采取最简单的保底方法：只要任一端点是内部节点，整条查询回退到完整原图。这样距离正确，但会浪费压缩索引，而且会让高频端点区域看起来非常差。

第二版阶段 0 改成“局部接入”：

- 内部起点只在所属区域内搜索到边界；
- 内部终点只在反向区域图内计算边界到终点；
- 再把边界距离作为本次查询的初始前沿接入压缩图；
- 如果起终点在同一区域，还额外比较区域内部直达距离。

这部分会在 05：精确端点局部接入 中详细解释。

13. 为什么“压缩更多”不一定更好

区域压缩存在多种成本和风险：

- 区域越多，索引预处理和存储通常越大；
- 边界越多，shortcut 可能呈平方增长；
- 区域过大时，局部端点接入本身可能需要展开较多节点；
- 多个高价值候选可能重叠，不能全部部署；
- 一个区域对某些 OD 很有用，对其他 OD 几乎无用；

- 毫无需求经过的区域即使能压缩，也不会贡献多少整体收益。

所以项目不是简单地“把全图都压缩”，而是在固定预算下选择最值得压缩的区域。

14. 什么是索引预算

预算表示允许索引占用的资源上限。可以用不同量度表示：

- shortcut 条数；
- 索引字节数；
- 被删除的内部节点数；
- 压缩图节点数和边数；
- 离线构建时间。

项目阶段三发现“请求 100 个区域”不是可靠预算，因为受不重叠约束影响，算法未必真能生成那么多区域。因此公平比较优先看实际 shortcut 数，同时报告内部节点和压缩图规模。

第一版主要使用约 38,000 至 40,000 条 shortcut 作为公平预算。第二版当前的单区域标签阶段暂时不做最终集合预算优化：它先问清楚“每一个候选单独看有多大价值”。

15. 什么是工作量收益

一条查询在原图展开 1,000 个节点，在单区域压缩图总共展开 800 个节点，则工作量收益为：

$$1,000 - 800 = 200 \text{ 个展开节点}$$

这里的“总共展开”必须包含：

- 压缩图搜索展开；
- 起点或终点局部接入展开。

如果漏掉局部接入成本，内部端点多的区域会被虚假高估。第二版标签代码明确把两部分相加。

16. 在线毫秒为什么不是当前唯一标签

毫秒耗时受很多非算法因素影响：

- 操作系统调度；
- CPU 当前频率；
- 其他进程负载；
- Python 垃圾回收；
- 缓存冷热；
- 查询执行顺序。

展开节点数则由图、查询和算法决定，重复运行更稳定。因此第二版当前把平均展开节点差作为主标签，把毫秒耗时只作为辅助记录。最终方法仍必须做同进程、逐查询配对的真实耗时评测。

17. 本章对应的核心代码

- 图结构：src/graph_types.py
- 区域和边界：src/regions.py
- shortcut 与压缩图：src/compression_index.py
- 压缩图在线查询：src/indexed_query.py
- 精确双向 Dijkstra：src/shortest_path.py

下一章会解释：既然传统压缩已经成立，第一版 GNN 为什么仍然不足，以及第二版为什么必须把训练目标改成真实区域收益。